CMSI 4071: Pilone & Miles Book Assignment 2

Problem 1

Two major concerns of any software project are its overall cost and the time it takes to complete it. To us, both are crucial concerns because time can affect customer satisfaction and potential competitors so it is important to stay timely especially with deadlines; however, cost can also affect the progress and scale of the project because with no money, the project cannot be fully developed. Since we have to choose one, we feel that the time is the more important concern for software projects, because time affects the deadlines of the project and overall customer satisfaction. Also, not being timely or efficient enough can cause competitors to develop quicker and can even cause project delays that prevent the product from ever reaching full development. The idea of complete functionality is also important for both time and cost, because all good software needs to meet the customer's standards and requirements, so prioritizing important features along with a timely schedule and within a good budget leads to a successfully developed project.

Problem 2

Planning, requirements analysis, design, development, and testing are the five primary stages of an Agile iteration. The activities should still be reviewed in each iteration, but we think that some preliminary planning and requirements collection can be done at the start of the project. This is because requirements are subject to change based on consumer input. Since errors found later can take considerably longer to correct, we don't think skipping these stages would save time overall.

Problem 3

The main phases of the waterfall method are the requirements, design, implementation, testing, deployment, and maintenance phases. Waterfall completes each of the phases before moving onto the next one, while agile repeats development activities through shorter iterations rather than having distinct phases like waterfall. Waterfall emphasizes completing the requirements, documentation, and design in the beginning parts of development before moving forward while Agile allows certain stages to be integrated and revisited throughout the development process while waterfall is sequential. Waterfall also includes more separated integration, deployment, and maintenance phases compared to agile, but agile still has these just in a more continuous way rather than a specific step in the process. We think having Waterfall's phases is necessary in development, because the team needs to carefully plan each stage before moving forward

especially when things are going in a step by step order. There are specific situations where the “extra” stages waterfall has would be useful in Agile such as developing banking software where critical customer information is being used and there should probably be different deployment and security check phases to ensure sensitive information is protected.

Problem 4

1. What is a user story?

A user story is a short description of a feature or requirement that explains what the customer wants the software to do.

2. What is blueskying?

Blueskying is brainstorming with the customer to gather ideas and possible requirements without immediately worrying about limitations.

3. What are four things user stories SHOULD do?

User stories should describe what the customer wants, focus on a specific feature, be easy to understand, and provide enough information for developers to estimate the work.

4. What are three things user stories SHOULD NOT do?

User stories should not be a long essay, use technical terms that are unfamiliar to the customer, and mention specific technologies.

5. Does the Waterfall method have user stories?

Waterfall can use user stories, but it traditionally relies on more detailed requirements documents and use cases.

Problem 5

Statement 1: All assumptions are bad, and no assumption is a good assumption.

We slightly disagree because assumptions are sometimes necessary when developing software, and often come from technical standpoints from a developer’s view. However if assumptions are made, the customer should always be consulted to ensure that the assumption is correct. Assumptions should always be questioned and verified before development begins so that time isn’t wasted developing something that is undesired by the customer. A truly good assumption would only occur after verifying that it’s what the customer really wants.

Statement 2: A big user story estimate is a bad user story estimate.

We agree because a large estimate usually means the user story is too complicated or contains too many tasks. The book explains that estimates over 15 days allow too much room for error and are more likely to be inaccurate compared to estimates that are under that threshold. It also states that some people believe estimates longer than seven days should be double-checked. We think that breaking large user stories into smaller ones makes them easier to estimate, develop, and track.

Problem 6

1. You can dress me up as a use case for a formal occasion.

User story

2. The more of me there are, the clearer things become.

User story

3. I help you capture EVERYTHING.

Blueskying, Observation

4. I help you get more from the customer.

Role playing, Observation

5. In court, I'd be admissible as firsthand evidence.

Observation

6. Some people say I'm arrogant, but really I'm just about confidence.

Estimate

7. Everyone's involved when it comes to me.

Blueskying

Do you agree with the book answers?

We agree with the book's answers because each technique helps developers understand the customer's requirements. At first, we thought planning poker could also fit the last statement because the development team participates in estimating tasks. However, the book explains that customers are not involved in planning poker, so blueskying is the better answer.

Problem 7

A “better than best-case estimate” is when an estimate assumes the work will be completed faster than even the most optimistic realistic prediction. Usually, the unexpected problems , debugging, and other delays are ignored in this estimate. The textbook specifies that developers often think in ideal working days, but that is not realistic because real projects have interruptions and additional tasks that cannot be foreseen. We think estimates should account for these issues so that development teams can have a more realistic schedule, even if it is a slight overestimate of the actual timeline.

Problem 8

We think that the best time to tell a customer that the delivery schedule cannot be met is as soon as the team realizes there is a problem that greatly hinders the schedule. This gives the customer time to adjust expectations and discuss possible solutions. It would definitely be a difficult conversation because there’s always a risk of the customer being disappointed, especially when earlier dates were proposed. The conversation could be made easier by being honest, explaining the problem, and offering alternatives like reducing the scope or extending the deadline, all while ensuring that the customer will still be satisfied.

Problem 9

We think branching is useful in software development, but too much branching will create problems. The textbook explains that every branch creates another codebase that needs to be maintained, tested, and updated. For example, if our Stockd project had an older released version that needed a bug fix, we could create a separate branch to fix it without interfering with the current development version in case our solution goes haywire. However, unnecessary branches should be avoided because they can make the project harder to manage, especially when the branches are for things that don’t necessarily need their own split.

Problem 10

Yes, we have used build tools while developing software, including Expo's build tools for React Native applications. One thing we like about Expo is that it makes building and testing mobile applications easier without needing to configure everything manually. It also helps developers to test changes efficiently and quickly. One downside is that dependency issues and build errors can sometimes be really difficult to troubleshoot. However, overall we think that build tools are useful because they automate parts of the development process and make projects more manageable.